

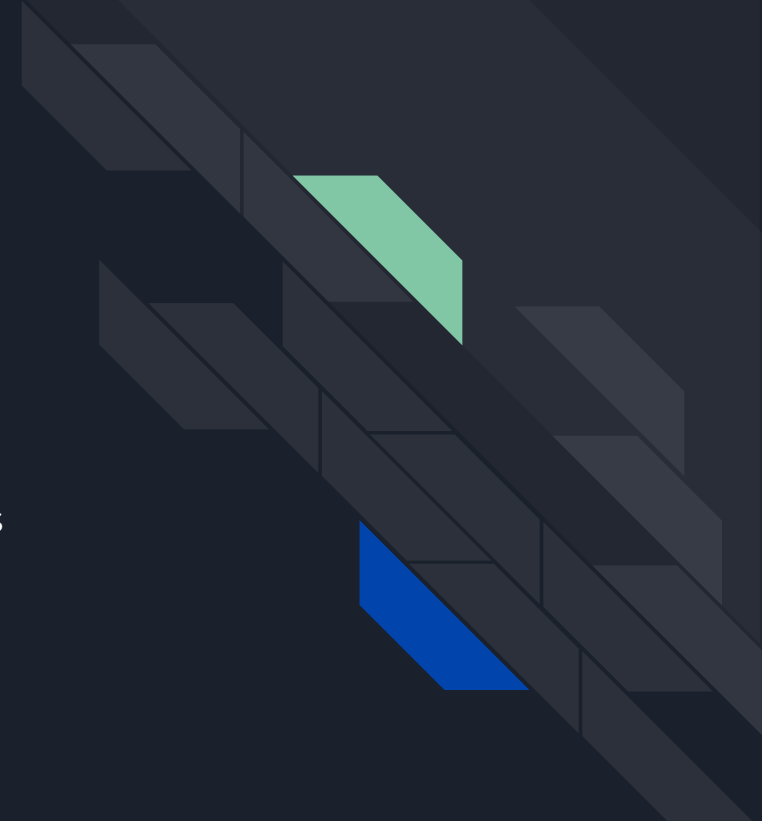


SmartRoom

Alexandre LASSAIGNE | Bastien RECORD

Sommaire

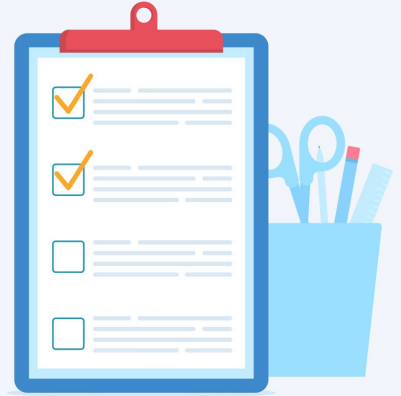
1. Présentation du Projet
2. Architecture UML
3. Choix des Technologies
4. Twelve-Factor
5. Organisation du Travail
6. Difficultés Rencontrées
7. Fonctionnalités Développées
8. Améliorations Possibles
9. Démonstration



1. Présentation du Projet

Cahier Des Charges :

- Informatiser le suivi de l'ensemble des salles :
 - Améliorer la réservation de salles
 - Avoir une traçabilité des réservations des salles
 - Avoir une vision d'ensemble des réservations
- L'application doit permettre :
 - Situer l'emplacement des salles
 - Consulter les disponibilités des salles
 - Rechercher et réserver des salles
 - Connaître les équipements des salles
 - Afficher le taux d'utilisation





2. Architecture UML



[Diagramme des Cas d'Utilisation](#)

[Diagramme de Domaine](#)

[Diagramme de Classe](#)

[Diagramme de Séquence Utilisateur](#)

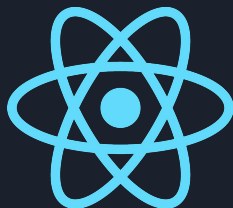
[Diagramme de Composant](#)

[Diagramme de Déploiement](#)

≡ 3. Choix des Technologies



NestJS



React



PostgreSQL



Bootstrap



Docker



4. Twelve-Factor

1. Base de code

GitHub => SmartRoom

SmartRoom

- |
- |— architecture
- |
- |— backend
- |
- |— frontend
- |
- |— db

2. Dépendances

package.json

3. Configuration

Variable d'environnement
dans docker-compose.yml



4. Twelve-Factor



4. Services Externes

Base de Données PostgreSQL



API NestJS (Gateway, JWT)

5. Build, Release, Run

CI Backend

=> Documentation

=> Test & Prettier

CI Frontend

=> Prettier

6. Processus

Stockage dans PostgreSQL

Pas de stockage/cache
navigateur



4. Twelve-Factor

7. Liaison de port

Frontend

=> PORT 5173

Backend

=> Gateway 3000

=> Services : 3001-3007

PostgreSQL

=> PORT 5432

8. Concurrence

Processus indépendants

9. Jetable

Démarrage :

docker compose up -d

Arrêt :

docker compose down



4. Twelve-Factor

10. Parité Dév/Prod

Déploiement de l'application
avec *docker-compose.yml*

11. Logs

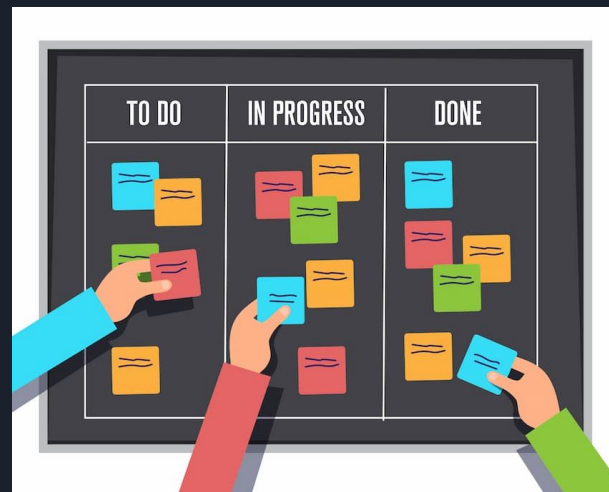


12. Administration

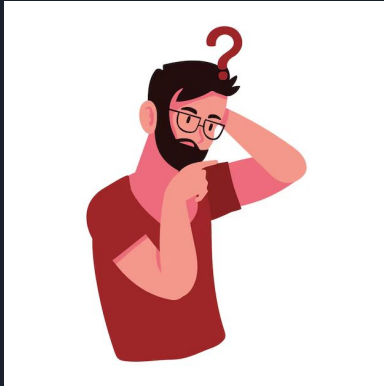


≡ 5. Organisation du Travail

- Système de ticket dans un projet GitHub
- Bastien :
 - Docker
 - Backend
 - Connexion Backend - Frontend
- Alexandre :
 - Backend
 - Design frontend



≡ 6. Difficultés Rencontrées



- Se familiariser avec l'architecture micro-service
- Mettre en place le JWT
- Utiliser Bootstrap
- Reproduire le design de la maquette
- Gestion des conflits

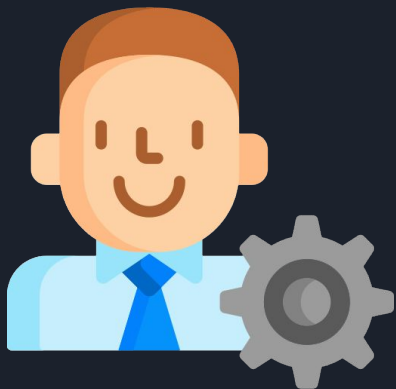
≡ 7. Fonctionnalités Développées

User :

- Calendrier permettant une vision d'ensemble des réservations
- Réservation d'une salle
- Visualisation de nos réservations futur et passées
- Création d'une demande ou d'un incident à l'administrateur
- Modification de ses coordonnées



≡ 7. Fonctionnalités Développées



Admin :

- Même fonctionnalités que l'utilisateur
- Gestion des utilisateurs
- Création/Modification d'un bâtiment / salle
- Vision global du taux de réservation et d'occupation
- Traitement des demandes et incidents

≡ 8. Améliorations Possibles

- Modifier une réservation
- Rappels d'événements
- Notification utilisateur
- Notification administrateur

